

Applications: LLMs in Health and Social Science Research



DANIEL VALDENEGRO

*Leverhulme Centre for Demographic Science & Nuffield
College
University of Oxford*

Large Language Models for Health and Social Science
Research, June-July 2026

Classical ML in NLP: the baseline

Before large language models, most NLP systems were built as **statistical learning** systems: they estimated patterns from text and used those patterns for prediction, extraction, or retrieval.

A standard learning problem is

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad f_\theta : \mathcal{X} \rightarrow \mathcal{Y},$$

where x_i is an input, y_i is a target, and θ are model parameters.

The training objective is usually written as

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i) + \lambda \Omega(\theta),$$

with a loss function ℓ and regulariser Ω .

Key idea: classical NLP learns useful mappings from text to labels, structure, or latent representations.

Supervised learning: labelled prediction

In supervised learning, the learner observes labelled examples (x_i, y_i) and fits a predictive model.

A probabilistic classifier often models

$$p_{\theta}(y \mid x), \quad \hat{y} = \arg \max_y p_{\theta}(y \mid x).$$

Common task types:

- ▶ **Classification:** $y \in \{1, \dots, K\}$
- ▶ **Regression:** $y \in \mathbb{R}$
- ▶ **Structured prediction:** y is a sequence or structured object

The target label acts as a compressed summary of the phenomenon of interest: sentiment, diagnosis, topic, stance, risk, relevance.

In NLP, supervised learning is used whenever labelled text is available.

Typical applications include:

- ▶ document classification;
- ▶ sentiment analysis;
- ▶ topic or intent classification;
- ▶ named entity recognition;
- ▶ question answering;

These tasks usually reduce a complex text input to a finite label, span, score, or structured output.

That reduction is useful, but it is also a form of loss: the model keeps what matters for the task and discards the rest.

Unsupervised learning: structure without labels

Unsupervised learning uses only observed inputs $x_1, \dots, x_n$, without target labels.

The goal is to discover latent structure in the data:

- ▶ clustering;
- ▶ dimensionality reduction;
- ▶ density estimation;
- ▶ topic modelling;
- ▶ latent factor learning.

A generic formulation is

$$\{x_i\}_{i=1}^n \longrightarrow z_i \quad \text{or} \quad p_\theta(x),$$

where z_i is a latent representation or $p_\theta(x)$ is a model of the data distribution.

The learner is not trying to predict a known label; it is trying to represent the regularities already present in the text.

Unsupervised learning in NLP: typical applications

In NLP, unsupervised methods are used to explore text when labels are absent, expensive, or unreliable.

Typical applications include:

- ▶ topic discovery;
- ▶ document clustering;
- ▶ semantic similarity;
- ▶ anomaly detection;

Earlier NLP systems often relied on bag-of-words, co-occurrence counts, topic models, and embeddings.

These methods are useful because they compress text into a form that is easier to compute with, but the price is that some structure is inevitably lost.

The distinction is straightforward.

- ▶ **Supervised learning** learns from explicit targets.
- ▶ **Unsupervised learning** learns from structure in the input alone.

Both are forms of representation learning: they transform text into a more manageable form.

This is the historical bridge to LLMs. Modern language models inherit the same basic pipeline, but they add **self-supervision**: the training signal is generated from the text itself, rather than manually annotated labels.

So the real transition is not from “statistics” to “intelligence”. It is from hand-built features and small models to learned representations at scale.

The LLM objective: next-token prediction

Modern decoder-only language models are trained on a simple prediction task: given a prefix $x_{<t} = (x_1, \dots, x_{t-1})$, predict the next token x_t .

For a sequence $x_{1:T}$, the model defines

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}).$$

Training maximises the log-likelihood of the observed text:

$$\hat{\theta} = \arg \max_{\theta} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}).$$

This is a **prediction** problem in the classical sense: the model learns to estimate the most likely continuation from observed context.

Where next-token prediction fits in the learning framework

Next-token prediction is a form of supervised learning, but with the labels generated from the text itself.

The training pairs are:

$$x_{<t} \mapsto x_t,$$

so the model learns from its own input sequence rather than from external annotation.

This is why it is often called **self-supervised** learning:

- ▶ the supervision signal is automatic;
- ▶ the task is still predictive;
- ▶ the objective is still optimisation of a loss function.

The causal mask enforces the predictive structure: token t may attend only to tokens $< t$, not to future tokens. That is not a technical detail. It is the whole game.

Why scale matters: from better prediction to apparent higher-level ability

At small scale, next-token prediction looks like ordinary language modelling. At large scale, the same objective can produce much richer-seeming behaviour.

As model size, data scale, and training compute increase, the system becomes better at predicting plausible continuations in a wide range of contexts:

- ▶ instruction following;
- ▶ summarisation;
- ▶ question answering;
- ▶ reasoning-like behaviour;
- ▶ in-context learning.

The model is still doing next-token prediction. What changes is the quality and flexibility of that prediction, which can make the system appear to possess more abstract capabilities than the objective itself explicitly encodes.

Can Large Language Models Transform Computational Social Science?

The paper asks whether zero-shot LLMs can support computational social science tasks such as:

- ▶ classification of social phenomena;
- ▶ explanation and coding of text;
- ▶ downstream annotation workflows.

Setup. The authors evaluate **13 language models** on **25 representative English CSS benchmarks**, and propose prompting best practices together with a broad evaluation pipeline.

Main result. LLMs do *not* outperform the best fine-tuned models on taxonomic classification, but they can achieve fair agreement with humans and can generate useful free-form explanations.

Interpretation for this lecture.

This is not a claim that LLMs replace standard social science methods. It is a claim that they can sometimes act as:

- ▶ zero-shot annotators;
- ▶ assistants in coding pipelines;
- ▶ generators for harder open-ended tasks.

Reflection question. When does an LLM output count as a useful measurement of a social phenomenon, and when is it merely a plausible textual proxy?

The interesting question is not whether the model can produce a label or explanation. It is whether that output is reliable enough to support inference in actual research.

Context windows: the model only sees a finite prefix

A modern LLM does not condition on the whole world. It conditions on a **context window**: a finite sequence of tokens available at inference time.

If the input is

$$x_{1:T} = (x_1, \dots, x_T),$$

then the model predicts each next token using only the tokens currently inside the window:

$$p_{\theta}(x_t \mid x_{<t}).$$

The context window contains everything the model can use at that moment:

- ▶ the task instruction;
- ▶ the user query;
- ▶ prior dialogue or retrieved text.

Key limitation: anything outside the window is invisible to the model. No memory, no magic, no telepathy. Just tokens.

How context windows enable zero-shot behaviour

Zero-shot behaviour means the model is asked to perform a task **without task-specific fine-tuning or examples**.

Instead, the task is specified in the prompt:

instruction + input $\longrightarrow$ output.

The model uses the context window to infer what kind of continuation is expected. For example, if the prompt says:

Classify this text as positive, negative, or neutral.

the model does not change its parameters; it simply conditions on the instruction and produces the most plausible label.

This is the bridge to the Ziems example: LLMs can act as zero-shot annotators because the task definition is placed inside the context window, and the pretrained model has learned a broad mapping from linguistic patterns to task-like continuations.

Prompt engineering: conditioning the model through context

Prompt engineering is the practice of shaping the context window so that the model produces the desired kind of output.

Useful prompt components include:

- ▶ explicit instructions;
- ▶ examples;
- ▶ labels or output schemas;
- ▶ constraints on length, style, or evidence.

Interpretation. Prompt engineering does not change the model weights. It changes the conditional distribution by changing the context. That is why zero-shot performance can vary sharply with wording, examples, and framing.

Prompting versus fine-tuning

A pretrained LLM already contains a broad statistical model of language. At inference time, we can adapt its behaviour in two main ways:

- ▶ **Prompting:** change the context, not the parameters.
- ▶ **Fine-tuning:** change the parameters using task-specific data.

Formally, prompting works by conditioning the existing model:

$$p_{\theta}(y \mid x, c),$$

where c is the prompt or instruction.

Fine-tuning changes the model itself:

$$\theta \rightarrow \theta',$$

usually by optimising on labelled examples from a specific task.

Practical distinction: prompting adapts the input; fine-tuning adapts the model.

Fine-tuning as task-specific adaptation

Fine-tuning starts from a pretrained model and continues training on a narrower dataset.

If the pretrained model has parameters θ_0 , then task adaptation estimates

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i),$$

often starting from θ_0 rather than random initialisation.

This gives the model:

- ▶ stronger performance on the target task;
- ▶ more stable formatting;
- ▶ less sensitivity to prompt wording;
- ▶ but also greater dependence on task-specific data.

In NLP this was the standard route for classification, tagging, and extraction before LLM prompting became dominant.

How the two approaches differ

Prompting

- ▶ no parameter update;
- ▶ fast and cheap;
- ▶ flexible and general;
- ▶ performance depends strongly on wording and examples.

Fine-tuning

- ▶ updates model weights;
- ▶ requires task data and compute;
- ▶ often better for stable, repetitive tasks;
- ▶ less brittle once trained, but narrower in scope.

Supervised classification vs. zero-shot LLM classification

The main distinction is the **loss function**.

Traditional supervised classification

$$\mathcal{L}_{\text{sup}} = \ell(f_{\theta}(x_i), y_i),$$

where y_i is an explicit gold-standard label and the model is trained to minimise task-specific error.

Zero-shot LLM classification

$$\mathcal{L}_{\text{pretrain}} = - \sum_t \log p_{\theta}(x_t \mid x_{<t}),$$

where the model was trained long before on next-token prediction, not on the downstream classification task.

So in zero-shot use, there is no task-specific training loss at all. The label-like output is produced by **test-time conditioning** on the prompt and input text.

Bottom line: classical classifiers learn the task directly; LLMs acquire the relevant behaviour indirectly during pretraining.

ChatGPT Outperforms Crowd-Workers for Text-Annotation Tasks

The paper studies a very practical question in applied NLP: can a pretrained LLM be used as a **zero-shot annotator** for text classification tasks?

Empirical setting. The authors evaluate ChatGPT on four datasets of tweets and news articles, with a total of 6,183 texts. The annotation tasks include:

- ▶ relevance detection,
- ▶ stance classification,
- ▶ topic classification,
- ▶ frame detection.

Comparison. ChatGPT is compared against MTurk crowd-workers and trained annotators using the same codebooks and annotation instructions.

The headline result is simple: ChatGPT performs surprisingly well as an annotator, even without task-specific training.

Main findings.

- ▶ ChatGPT's zero-shot accuracy exceeds MTurk by about 25 percentage points on average.
- ▶ ChatGPT's intercoder agreement is higher than both crowd-workers and trained annotators.
- ▶ Lower temperature makes outputs more consistent, with temperature 0.2 producing especially high agreement.
- ▶ The per-annotation cost is below \$0.003, making it dramatically cheaper than MTurk.

The interesting point is not only that the model is accurate. It is that it can replace or support parts of the annotation pipeline.

Potential uses:

- ▶ rapid labelling of large corpora;
- ▶ pre-annotation before human review;
- ▶ exploratory coding when no labelled data exist.

But the result should not be read naively.

Important caveat. The tasks are relatively well-specified, the label sets are finite, and the evaluation is still against a human-defined coding scheme. That means the model is learning to imitate a research protocol, not discovering social reality in the abstract.

What does success mean here?

If an LLM produces labels faster and cheaper than crowd-workers, does that mean it is measuring the phenomenon better, or only that it is approximating the annotation rule more efficiently?

- ▶ When is zero-shot annotation good enough for substantive research?
- ▶ Does higher agreement necessarily imply higher validity?
- ▶ What happens when the target concept is fuzzy, contested, or culturally dependent?
- ▶ At what point does the model become a convenience layer rather than an instrument of inference?

In-context learning: learning from the prompt

In-context learning means that the model performs a task from examples or instructions *inside the prompt*, without any parameter update.

A prompt may contain:

- ▶ an instruction;
- ▶ one or more worked examples;
- ▶ the input instance to be completed.

Schematically:

$$(x_1, y_1), \dots, (x_k, y_k), x_{k+1} \longrightarrow y_{k+1}.$$

The model does not retrain itself on the examples. It uses them as contextual evidence for what kind of output is expected.

Why in-context learning matters

In-context learning is one of the most useful features of LLMs because it lets the same model switch tasks without retraining.

For example, the prompt can specify:

- ▶ a label set;
- ▶ the output format;
- ▶ an annotation rule;
- ▶ a style or level of detail.

So the model behaves as if it has been adapted to the task, but the adaptation happens through context rather than training.

10 min break

Why evaluation is its own problem

A model is only as useful as our ability to know how well it works.

Evaluation answers two different questions that are easy to conflate:

- ▶ **Reliability:** does the model behave consistently?
- ▶ **Validity:** does the model measure what we actually care about?

A model can be highly reliable and still invalid: consistently wrong is still wrong.

Key idea: good evaluation design separates “does it agree with itself / with humans” from “does it correspond to the underlying phenomenon”.

Train, validation, and test: why splits exist

The basic problem: a model that has seen the data will look better than it really is.

Standard practice splits $\mathcal{D}$ into disjoint subsets:

$$\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}} = \mathcal{D}, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset.$$

- ▶ **Train:** fit θ ;
- ▶ **Validation:** tune hyperparameters, select models;
- ▶ **Test:** report performance, touched only once.

For LLMs specifically: pretraining corpora are enormous and often undisclosed. If benchmark text overlapped with pretraining data, “zero-shot” performance is partly **memorisation**, not generalisation. This is a live, unresolved problem in LLM evaluation, not a solved technicality.

For binary or multiclass classification, the confusion matrix is the basic unit of evaluation.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Every aggregate metric below is a different summary of this table. The choice of metric is a choice about **which errors you care about most**.

Accuracy, precision, recall, F1

Accuracy: overall correctness.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Misleading under class imbalance: a classifier that always predicts the majority class can score high accuracy while being useless.

Precision: of predicted positives, how many are correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: of actual positives, how many were found.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1: harmonic mean, balances the two.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Agreement metrics: do raters agree more than chance?

Accuracy against a single gold label assumes the gold label is unambiguous. In CSS and health research it often isn't, so we also measure **agreement** between coders, or between a coder and the model.

Cohen's kappa corrects raw agreement for the agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement and p_e is expected agreement under independence.

$\kappa = 1$: perfect agreement. $\kappa = 0$: chance-level agreement. $\kappa < 0$: worse than chance.

Connecting back to Gilardi et al.: “ChatGPT’s intercoder agreement is higher than crowd-workers” is fundamentally a κ -type claim. High agreement tells you the model is *consistent* with a coding scheme. It does not by itself tell you the coding scheme was correctly specified, or that the underlying construct (stance, frame, relevance) was well captured.

Perplexity: evaluating the language model itself

Classification metrics evaluate a downstream task. **Perplexity** evaluates the language model's own predictive distribution directly.

For a held-out sequence $x_{1:T}$,

$$\text{PPL}(x_{1:T}) = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}) \right).$$

- ▶ Lower perplexity means the model assigns higher probability to the actual continuation;
- ▶ it is a direct readout of the next-token training objective, not of any downstream task;
- ▶ a model can have excellent perplexity and still perform poorly on a specific classification or annotation task, because perplexity says nothing about task alignment.

Validity: the question metrics don't answer

None of the metrics so far tell you whether the label, score, or output corresponds to the phenomenon you actually want to measure.

Classical measurement theory distinguishes:

- ▶ **Reliability:** consistency of measurement (captured by accuracy, κ , F1);
- ▶ **Construct validity:** does the measurement correspond to the theoretical concept it claims to capture?

Example. A model can classify “populist rhetoric” or “clinical distress” with high inter-rater agreement against a codebook, while the codebook itself only weakly captures the contested or context-dependent concept it was designed to operationalise.

Practical consequence: agreement-based metrics (including most LLM annotation benchmarks) are reliability evidence. Validity requires an independent argument: theoretical justification, external criterion validation, or triangulation with another measure.

Some outputs (summaries, explanations, open-ended generation) have no single correct label, so automatic metrics are insufficient.

Human evaluation: raters score outputs on dimensions such as fluency, factuality, helpfulness, or harm. Expensive, slow, but treated as the gold standard.

LLM-as-judge: using a (typically stronger) LLM to score another model's output, as a cheaper proxy for human evaluation.

Caution: this reintroduces the same reliability/validity gap one level up. If the judge model has systematic biases (favouring longer answers, particular styles, or its own family's outputs), agreement with the judge measures conformity to the judge, not ground truth.

- ▶ **Is the model internally consistent?** → accuracy, F1, Cohen's κ
- ▶ **Is the model a good language model?** → perplexity
- ▶ **Does the model's output correspond to the real phenomenon?** → no single metric; requires construct validity argument, external validation, or triangulation
- ▶ **Is open-ended output good?** → human evaluation, or LLM-as-judge with awareness of its own biases

Bridge forward: this is exactly the gap the reflection questions from Case study 2 were pointing at. Reporting a high κ between ChatGPT and human coders is a reliability claim. Whether that translates into valid social-scientific measurement is a separate, harder question that metrics alone cannot settle.

Summarisation as a second encoding step

Summarisation takes a long text O and produces a shorter one $T(O)$:

$$S \longrightarrow O \longrightarrow T(O),$$

exactly the chain from Lecture 1. By the Data Processing Inequality, $I(S; T(O)) \leq I(S; O)$: a summary cannot contain more about the original phenomenon than the source text already carried.

- ▶ **Extractive:** select existing spans from O ;
- ▶ **Abstractive:** generate new text conditioned on O , via $p_{\theta}(x_t \mid x_{<t}, O)$.

Health/social science use: clinical notes, long survey responses, policy documents, parliamentary debate.

Evaluating summaries: ROUGE and its limits

ROUGE measures n-gram overlap between a generated summary and a reference summary (recall-oriented).

- ▶ Easy to compute, widely reported;
- ▶ but overlap is not the same as correctness: a summary can score well on ROUGE while inverting or inventing a fact, because n-gram match says nothing about truth-conditions.

Bridge: this is the gap between O (what was said) and $T(O)$ (what the model says was said). High lexical overlap does not guarantee $T(O)$ preserves the *content*, only the *surface*. This is where hallucination becomes a measurable problem, not just a philosophical one.

From free text to structured fields

Information extraction maps unstructured text to a structured output: named entities, relations, or schema-conforming records.

“Patient prescribed 500mg metformin twice daily” $\longrightarrow$ {drug, dose, frequency}.

- ▶ NER: spans $\rightarrow$ entity types (person, organisation, date, diagnosis);
- ▶ relation extraction: structured tuples (actor, action, target);
- ▶ LLMs are increasingly asked to emit JSON directly, conditioned by prompt schema rather than fine-tuned output heads.

Practical failure mode: schema adherence is not guaranteed. An LLM may extract correctly but format incorrectly, which is a different failure from hallucination, and matters a great deal for downstream pipelines (e.g. parliamentary or clinical record databases).

What hallucination is, and why it follows from the objective

Hallucination: fluent, confident output that is unsupported by the source or by fact.

Recall the training objective from earlier:

$$\hat{\theta} = \arg \max_{\theta} \sum_t \log p_{\theta}(x_t \mid x_{<t}).$$

The model is optimised for **plausibility**, not truth. Nothing in this objective distinguishes a likely-sounding continuation from a true one.

Two kinds:

- ▶ **Intrinsic:** contradicts the given source/context;
- ▶ **Extrinsic:** unsupported by source, may or may not be true.

Key point: hallucination is not a bug bolted onto next-token prediction. It is what happens when plausibility and truth come apart, and the objective only ever optimised for the former.

A formal semantics angle

One way to sharpen this: next-token prediction trains a model over *surface form*, not over a model-theoretic interpretation function · mapping sentences to truth conditions.

- ▶ The model has no explicit notion of reference, satisfaction, or a world to check claims against;
- ▶ what it has is a learned distribution over what tends to follow what, given the corpus;
- ▶ fluency is a property of $p_{\theta}(x_t \mid x_{<t})$; truth is a property of a sentence's relation to the world.

Why this matters here: a model can be excellent at the first and indifferent to the second, by construction, not by failure.

Failure modes beyond hallucination

- ▶ **Prompt brittleness:** performance shifts with wording, framing, example order (Slide 15);
- ▶ **Inherited bias:** pretraining corpora carry social and demographic skew, which propagates into classification/extraction;
- ▶ **Inconsistency:** repeated runs at higher temperature diverge (recall Gilardi et al.'s $\kappa = 0.2$ finding);
- ▶ **Sycophancy:** the model tends to agree with or validate the framing in the prompt, a serious risk for annotation tasks where the prompt itself can prime the answer;
- ▶ **Context degradation:** information placed in the middle of long contexts is used less reliably than information at the edges.

Hallucination is confident fabrication, not uncertainty

A model that says “I don’t know” or hedges appropriately is not hallucinating.

Hallucination specifically means: fluent, confident output presented as fact, that is unsupported or false.

- ▶ The training objective $p_{\theta}(x_t \mid x_{<t})$ optimises for plausible continuation, not for calibrated expression of uncertainty;
- ▶ so a model can be *wrong and confident* far more easily than it can be *wrong and appropriately hesitant*;
- ▶ this is precisely why measuring whether a model’s expressed confidence tracks its actual reliability is an open and active research question, not a solved one.

The practical question is not “did the model get it right”, but “did the model’s confidence match how right it was”.

So what do you actually do about it

None of this means *don't use these tools*. It means knowing which failure mode you are exposed to, and putting a check in front of it.

- ▶ **Ground the model:** give it the source text directly (retrieval, context injection) rather than relying on parametric memory alone;
- ▶ **Human-in-the-loop:** keep a human review step for anything consequential — diagnosis-adjacent, publication-adjacent, policy-adjacent;
- ▶ **Lower temperature for consistency tasks:** recall Gilardi et al.'s finding that $T = 0.2$ produced especially high agreement;
- ▶ **Validate structured output:** re-prompt or programmatically check schema-conforming output rather than trusting it blindly;
- ▶ **Match the metric to the question:** reliability metrics (κ , F1) are not validity evidence; treat them accordingly.

Closing thought. Know which failure mode you are exposed to, and put a check in front of it.

Closing the loop

Summarisation, extraction, and classification all share the same exposure: each is a transformation $T(O)$ of an already-encoded source O .

- ▶ Summarisation can fabricate content;
- ▶ extraction can fabricate fields, or simply fail the schema;
- ▶ classification can agree with itself (κ) without being valid.

Reflection. None of this is fixed by a bigger model alone, because the Data Processing Inequality is not an engineering limitation; it is a structural one. The question for your own research is not whether the model is fluent, but what $I(S; T(O))$ actually survived the pipeline you built.

Reproducibility: LLMs are moving targets

Classical statistical models are fixed once estimated. LLMs accessed via API are not.

- ▶ Model versions are deprecated, silently updated, or retired without notice;
- ▶ even at temperature 0, outputs are not guaranteed deterministic across calls, hardware, or time;
- ▶ “we used GPT-4” is not a fixed method statement the way “we fit a logistic regression” is — the underlying p_θ you queried six months ago may not exist in that form today.

Consequence for research practice: this is a genuine threat to replication, not a footnote. If your method depends on a specific model behaviour, that behaviour itself needs to be treated as a variable to report and, where possible, stress-tested across versions and settings — the same robustness logic that motivates multiverse-style sensitivity analysis applies directly here.

Multilingual and cross-cultural validity

Pretraining corpora are overwhelmingly English-dominant. This has direct consequences for any cross-national or non-English research.

- ▶ Classification accuracy, calibration, and fluency typically degrade for lower-resource languages relative to English;
- ▶ what counts as “neutral” framing, tone, or even a sensible answer can itself be culturally specific, and models trained mostly on English-language, Western-skewed text may default to those norms silently;
- ▶ this is not just a translation problem — a model can be fluent in a language while still importing assumptions calibrated to a different cultural context.

Practical implication. For demographic science specifically, where much of the most important work is explicitly cross-national or cross-cultural, this failure mode deserves the same scrutiny as the classification validity question from earlier: agreement with an English-trained model is not evidence the underlying construct was captured correctly in another linguistic or cultural setting.